Bauhaus-Universität Weimar

SPOT as a companion in AEC industry project sites

Project Report

Abhishek Dilip Patil,
Matriculation Nr. – 127126

Niraj Kishore Ladhe
Matriculation Nr. – 125815

Guides:

Prof. Dr.-Ing. Christian Koch
Ronny Helbing M.Sc.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background and Motivation

The AEC (Architecture, Engineering and Construction) industry is evolving in today's time with "Industry 4.0". Like other industries, there is a shift from document-centric delivery to data-centric, mode-driven workflows that interconnect design, construction and operations. BIM is at the core, which provides a common data environment enriched by open standards (e.g. IFC) and linked to schedules and costs (4D/5D simulations). Field technologies - laser scanning, drones, quadruped robots and IoT sensors - feed continuous reality capture back into the model, enabling automated progress verification, quality checks, and safety monitoring. Structured data, APIs and cloud collaboration, teams move from periodic reporting to near-real-time updates with dashboards and predictive control, where deviations are detected early, and targeted interventions and design updates are timely propagated downstream. A lot of benefits have been observed with this change. It includes fewer clashes and RFIs, reduced rework, faster inspections, and better traceability. Although, new challenges such as data governance, interoperability, cybersecurity and organizational change remain, digitalization has helped perform more complex projects with multiple stakeholder seamlessly and cost effectively. [1]

The current construction monitoring and inspection is manual process and has led to inefficient contributions to cost overruns in 66% of construction project and schedules delays by 53% in projects, affecting almost $1.4 trillion worths of structures being constructed each year in United States. [2]

The absence of real time information handicaps manager's ability to monitor schedule, cost and other performance indicators, which reduces their ability to detect or manage the variability and uncertainty inherent in project activities. A significant portion of the inspection time (30-50%) is spent on visual data collection of the as-built status and on-site analysis. [1] Also, the quality of inspection data is also determined by the inspector's experience and the methods used to collect measurements. [2]

Digitalization is the first part of shift in the way industry works. It opens the door to high level automation of tasks. Many repeatable tasks are tedious for humans and affect the precision and accuracy. Automation of such tasks can better help the professionals to divert their focus on analysis rather than data collection, helping organizations to take decisions faster. Robots with sensors and cameras mounted as payloads, can perform these tasks with higher precision and no fatigue. Also, robots can enter hazardous environments, and the operator can inspect the area without any risk. [3]

The above benefits motivated us to look at solutions that involved a quadrupled robot. Quadrupled robots are more versatile in a construction site due to the design being close to the natural evolution of mammals. Although, there are many case studies and research on the use of robots at construction sites. In autonomous operations, most of those applications have a limitation of only having a prerecorded path that a robot

can follow. While performing these operations, manual intervention in the movement is still felt necessary, due to the complex environment of the site. But the autonomous program should be completely exited to switch to manual mode which is not a user-friendly way and has high lead time. We wanted to come up with a solution that can bridge the gap where autonomous feature can keep on working but also accept manual intervention when felt necessary by the professionals. An indirect comparison to having a trained dog by the side to perform certain tasks on its own but also obey to some commands given by the trainer.

1.2 Problem Statement

The main challenge of this project is to make SPOT navigate by actively interacting with the surrounding - specifically, by continuously scanning the environment for fiducial tags - and to use those detections to drive motion decisions instead of following a pre - defined path. This requires reliable tag detection and robust decision logic so SPOT can approach, confirm, and act on tags despite occlusions, motion or variable lighting. In parallel, a layer of manual oversight must always be available, with access to features ranging from emergency stop to movement of SPOT. Thus, an operator can intervene safely and intentionally at any time. Implementation of these features requires clear understanding of the basic operation of SPOT, communication pathway between SPOT and the client system, including how commands, status and feedback are exchanged and synchronized. It also requires gaining knowledge on the various payloads available for SPOT for better operation and data handling. Equally important is proper handling of the errors and response signals to avoid system crashes. Also looking for proper open-source libraries compatible with the SPOT API to fill the gaps needed for functioning of the code to ensure stable integration and maintainable code.

1.3 Objectives

The objectives of this project are to design a control stack for a navigation system through a construction site with the help of fiducial tags by identifying each tag distinctly and assigning features to some tag IDs. The next feature to be implemented is that of having manual input for navigation in tight spaces, where the autonomous system can get stuck due to obstacle detection parameters and blind spots in the robot's visible region. The implementation of both the different types of control is done in a way where the manual input can override and take priority over the autonomous functioning. Finally, a user feature of able to see the view of the robot is implemented to better understand the situation from the robot's perspective which can help to take further decisions. The objectives are then divided in technical tasks as mentioned below:

1. SPOT program with both autonomous and manual functionality:

   Integrating the manual functionality with fiducial detection is a challenge as simultaneous access can lead to runaway between requests, RPC timeout errors or erratic behaviour from SPOT. Also, heavy logging overloads the network which can again lead to laggy response. A seamless integration of senor data transfer and essential command signals for robust performance. Keeping a basic but user-friendly interface for clear understanding of the state of SPOT and better handling. And at last, using minimal computation power while autonomous functioning is important for lag-free operation.

2. Main SPOT manager class:

   Creating a main class that has control over the essential features like the emergency electronic stop (ESTOP), motor power, establishing valid network connection between the SPOT and the client system, verifying that the connection is secure. Then, along with the essential services, also having decent manual control on the movement of the SPOT for manual navigation through tight spaces where the parameters set for collision detection might fail. Having a basic user interface to know the mode of the robot or see important information, e.g. battery percentage, status of motor power, network connection validity.

3. Fiducial tag detection, follow and react to different tag IDs:

   A secondary class having features for fiducial detection using the body cameras present on SPOT and following the tag specified by the user. SPOT should be able to detect all the fiducial tags present in the detection range. Assigning a top priority to tag (master tag) where the SPOT will follow the tag if detected. Reacting to another tag ID (preferably a physical movement).

4. Video Stream and priority to manual input:

   Stream the video from all the available cameras of SPOT for operation in places where it is difficult to have SPOT in the range of vision. Ensuring the stream is not lagging the actual movement by a large margin. This feature also helps to compare if a tag is in the visible range of SPOT.

   Coding the program for manual input to have a higher priority over fiducial follow for safe recoveries of SPOT from difficult surroundings.

2 Literature Review

Previous studies investigated automating construction inspection. The studies focused on improving how often and how accurately inspections occur by using automated systems to capture visual data. These studies either focused primarily on the data capture alone and not on presenting the information in a way that is intuitive for inspection or used heavy post-processing that requires skilled personnel with specialized training. For progress monitoring, accurate, timely, and regular data collection and analysis during the construction process is key. In fact, automated progress monitoring of construction projects using emerging technologies can address existing limitations and inefficiencies of the manual processes and can enable systematic recording and analyzing construction progress. Identified four areas in automating the process of progress monitoring: (a) data collection from the actual construction work, (b) extracting required as-built data for analysis, (c)comparing the as-built data with as planned information and (d) visualizing and reporting the results. Halder et al. presented "A Methodology for BIM-enabled Automated Reality Capture in Construction Inspection with Quadruped Robots" discussed on the above points. [4]

The study again went on to show that specifically in order to automate data collection in this process, various technologies, for example, a combination of Radio-Frequency Identification (RFID), laser scanning, photogrammetry with a digital camera, barcoding and a mobile device, a combination of camera-equipped UAV (Unmanned Aerial Vehicles), 4D Building Information Modeling (BIM), and 3D point cloud model and a data fusion framework that combines multiple data sources including 3D CAD model, ultra-wideband (UWB) receivers data, and laser scanned data. In this regard, collecting photo images of the actual construction status is very important for the reporting stage and for supporting communication of the identified issues with the stakeholders. 360° cameras are being utilized in these applications to increase visibility and decrease the capture time. Recent advances in automation have enabled robotic and autonomous inspection and monitoring. Mobile robots can be deployed to release the human from the laborious task of constant on-site image capture. [4]

Autonomous navigation through the unstructured and dynamically changing construction environments requires mobile robots that can carry out high-performance locomotion tasks. Mobile robots use various types of locomotion to navigate an environment. Typical locomotion types used for mobile robots are : (a) wheeled robots using multiple wheels for navigation as the most stable robots with high payload capacity but they can operate on limited uneven surfaces and cannot move between levels through the stairs independently, (b) crawler-mounted robots using crawlers with continuous tread bases that can operate on unlevelled and unfinished ground, (c) climbing robots using suction mounts or magnetic adhesion to stick to walls that are typically used for façade inspection but their challenge is to stay affixed to the surface reliably while being able to avoid obstacles, (d) Unmanned Marine Vehicles (UMV) mechanically sealed to operate in water and typically used for underwater inspection with special sensing devices, (e) Unmanned Aerial Vehicles (UAV) using rotors to fly and more suited for outdoors but in an enclosed construction environment there is a potential safety risk for workers and they may even collide with building materials, columns, or overhead cables, (f) legged robots using one or multiple legs inspired from terrestrial mammals to move but maintaining balance while moving is a major challenge for legged robots, (g) hybrid robots using a combination of locomotion systems e.g. a leg-wheel robot. Halder et al. presented

“Fundamentals and Prospects of Four-Legged Robot Application in Construction Progress Monitoring” talking on the use of various robots in AEC industry. [5]

The study also shows that legged robots improved the accessibility of various terrains and are more versatile than wheeled robots, demonstrating good potential for navigating construction sites. Many legged robots that have been developed and used are two-legged robots (biped or humanoid), four-legged robots (quadruped) and six-legged robots (hexapod). Recently, legged robots have become a major research area in robotics. Many legged robotic systems have been produced by companies, universities, and independent groups (NASA, MIT, IIT, ETH, Boston Dynamics, Ghost Robotics, ANYbotics). However, due to a high technological challenge to build and control such robots, few teams have created legged robots that have a large application base outside of laboratory setting. Multi-legged designs tend to offer better mobility because they travel faster while utilizing less total energy, they can handle obstacles faster and more effectively, and their cost of transport is lower. Legged robots also have more adaptability to rough terrain and better mobility because of increased flexibility due to the large Degrees of Freedom available in the legs and the ability to move the body independently of the underlying terrain making legged robots a good potential traversing construction environments. [5]

Quadruped robots can access wide range of terrains and are more versatile than wheeled robots, including better mobility, stability, and flexibility due to having more Degrees of Freedom (DoF) per leg. Quadruped robots can traverse uneven terrains, move up and down stairs, and walk over small obstacles, suitable for the construction environment. Quadruped robots can assist humans on job sites in construction inspection and monitoring, but they also require periodic human intervention on unstructured and dynamically changing construction sites. Halder et al. “Construction inspection & monitoring with quadruped robots in future human-robot teaming: A preliminary study” focused on the above points. [2]

Various case studies presented by Boston Dynamics were also studied to understand the current use of Spot in the Construction Industry. The Hamburg Port Authority experimented with the Spot robot from Boston Dynamics onboard an autonomous structural inspection inside the 3.8-km Köhlbrand Bridge, a space that is dark, hazardous and confined for human access. The Spot robot was outfitted with an EAP package and LiDAR/RTC360 payloads, and the rental team iteratively trained the paths and routes so that Spot was able to navigate narrow passages, conduct inspections for days at a time while autonomously recharging, all the while feeding inspection points information into a digital twin for transformable and repeatable data capture. Overall, the project established the feasibility and robustness of autonomous operations for condition-based maintenance, decreased inspector risk to unsafe conditions, and improved consistency and documentation of inspections. [6]

Turner Construction reports significant improvements in efficiency gained by using Boston Dynamics' Spot with DroneDeploy's Robotics platform to document autonomous site documentation on larger data-centre projects. Spot performs scheduled routes overnight to capture and upload daily 360° imagery, and then have this backed up to the cloud, automatically mapped to site floor plans (using StructionSite) and spatially linked to existing aerial scans to generate a consistent, geolocated record of progress on the site. This workflow supports virtual walk-throughs that leverage time comparisons, (e.g., review "X-rays" behind the wall), and systematically push data back into the VDC/BIM pipeline to identify and flag deviations early

and reduce rework. The case study showed that this approach to ground-robotics produced an observation time-savings of greater than +95%, while providing greater consistency and coverage and the ability to remotely tele-operate Spot for targeted inspections. [7]

RATP, the operator of the metro in Paris, adopted Boston Dynamics' Spot—the "Perceval"—for inspecting various rail infrastructure that is hard-to-reach and dangerous (tunnels, galleries, viaducts), as part of their effort to reduce worker exposure and improve inspections without disruptions to trains during the day, improving quality of visual evidence. Spot was equipped with a PTZ 360° camera with IR and operated via mesh-networked radios. Even with the limited connectivity of long tunnels, Spot captured visual and thermal evidence that supports this program, which will grow to inspect 75–100 civil works routines and plans to eventually utilize autonomous laser-scanning of digital twins (in collage with robotic systems). The case exemplifies mobile robotics for repeatable, safe inspection of legacy transit systems. [8]

Virginia Tech investigators, in collaboration with Procon Consulting, conducted field testing of Spot on three ongoing construction projects on campus to investigate autonomous progress monitoring using 360° imagery. After teleoperated path setup, Spot travelled autonomous routes once a week to take photos and upload information to HoloBuilder. The study reported that Spot was more consistent in data collection than human photo capturing as well as possible time savings and safety gains for inspectors, but everything should be taken in the context of human explanations of the observations. The study signals that university job sites are real labs for the advancement of construction robotics research, and the next research gap indicated by the researchers, was adoption issues (legal/safety requirements, workflows for the team). [9]

ACCIONA employs Spot with Trimble's FieldLink integration to automate the process of laser scanning in hazardous tunnel projects to capture consistent 3D as-built data while also keeping personnel out of blast zones, gas pockets, or areas with heavy equipment. Early testing indicates that Spot's legged mobility and ~90-minute battery life lends themselves well to rough terrain and its long distances, which can help with scanning during downtime while feeding design-validation workflows that compare progress with BIM plans. While the program continues to aim for safer work, richer datasets and productivity gains, autonomous routines will progress from testing to production. [10]

Pomerleau automated site photo documentation by training Spot to autonomously walk predefined routes and take pictures at guaranteed locations for HoloBuilder. For a 500,000 ft² project requiring ~5,000 images to be taken weekly, the workflow took about 20 staff-hours of work per week out of the equation (although coverage and consistency improved so that utilizations of HoloBuilder for progress tracking often increased). [11]

Brasfield & Gorrie collaborated with Boston Dynamics and DroneDeploy to document 360° photos and data for an entire job site using Spot. The contractor utilized Spot's SDK to build a custom payload that included both 360° cameras and other environmental sensors, and on-board computing, as well as web apps-one for mission planning and the other for mission execution to run autonomous routes, collect imagery, and upload images and data to DroneDeploy's 360 Walkthrough for a centralized repository of time-sequenced documentation. This approach removed the tedious photo-collection work from a project

manager's or engineer's to-do list, improved both coverage and frequency of imagery capture on job sites with tough terrain, and included status data from operations (battery, fleet pose, logs) that become valuable for discussing operations at scale. The capability to monitor site documentations is in place, framing mobile robotics is a viable path for consistent and auditable methods to monitor project progress. [12]

As part of Boston Dynamics' Early Adopter Program, Foster + Partners utilized Spot in the Battersea Roof Gardens project to automate the repeatable, high-frequency reality capture (laser scans / imagery) to track progress against the design models, or "as-built" status. Reports from the practice's Applied R+D team indicated that data was completed quickly and with improved documentation consistency and safely capturing data in the highly dynamic building environment - leading to the conclusion that legged robots are a valid enhancement option for capture tools for digital-twin workflows and site QA / QC programs. [13]

The innovation team at Swinerton also utilized Spot for its legged mobility and for the opportunity to link the last mile of an automation solution for tracking the progress of a project, trained on repeatable routes based on data and site observations recordings in past site observations - freeing the staff to do other on-site work. In starting legged mobility work as well as the reality capture workflow, coverage increased and so did reliable capturing of photos/scans to create end to end pipeline from field data to reliable progress analytics. This experience showcased legged automation as a practical bridge of the BIM / VDC plans to on-site verification. [14]

3 Methodology

3.1 Theory and SDK Documentation

The first step was reading the documentation provided by Boston Dynamics. The SDK documentation explains [15]:

1. **Conceptual documentation:** These documents explain the key abstractions used by the Spot API. It helps to get familiar with the various terms used in the documentation for further development.
2. **Python client library:** Applications using the Python library can control Spot and read sensor and health information from Spot. A wide variety of example programs and a QuickStart guide are also included.
3. **Payload developer documentation:** Payloads add additional sensing, communication, and control capabilities beyond what the base platform provides. The Payload ICD covers the mechanical, electrical, and software interfaces that Spot supports.
4. **SPOT API protocol definition:** This reference guide covers the details of the protocol applications used to communicate to Spot. Application developers who wish to use a language other than Python can implement clients that speak the protocol.
5. **Spot SDK Repository:** The GitHub repo where all the Spot SDK code is hosted by Boston Dynamics.

Understanding at least the main concepts related to SPOT is very important to further research with it. E.g. Networking, E-Stop, KeepAlive, Lease, Autonomy services. Also, the SDK is maintained by Boston Dynamics and there will be updates in the future as research progresses.

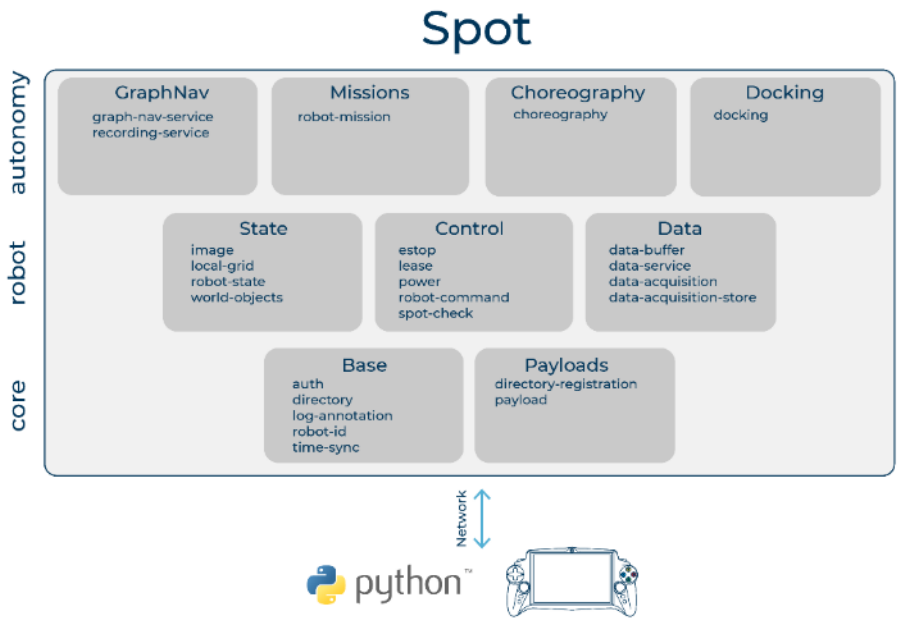


Figure 1: Block Diagram showing three different level of robot's services. Core, robot and autonomy. The Base, Payloads, Control and Data blocks are the necessary services that are a must to connect with SPOT. [15]

3.2 Communication and Networking

SPOT can be connected in both wired and wireless mode, with feasibility of both being an access point or a client in WiFi mode or even via custom communication links. gRPC is the application-level protocol used by most of the SPOT API. The open-source RPC framework developed by google. [16]

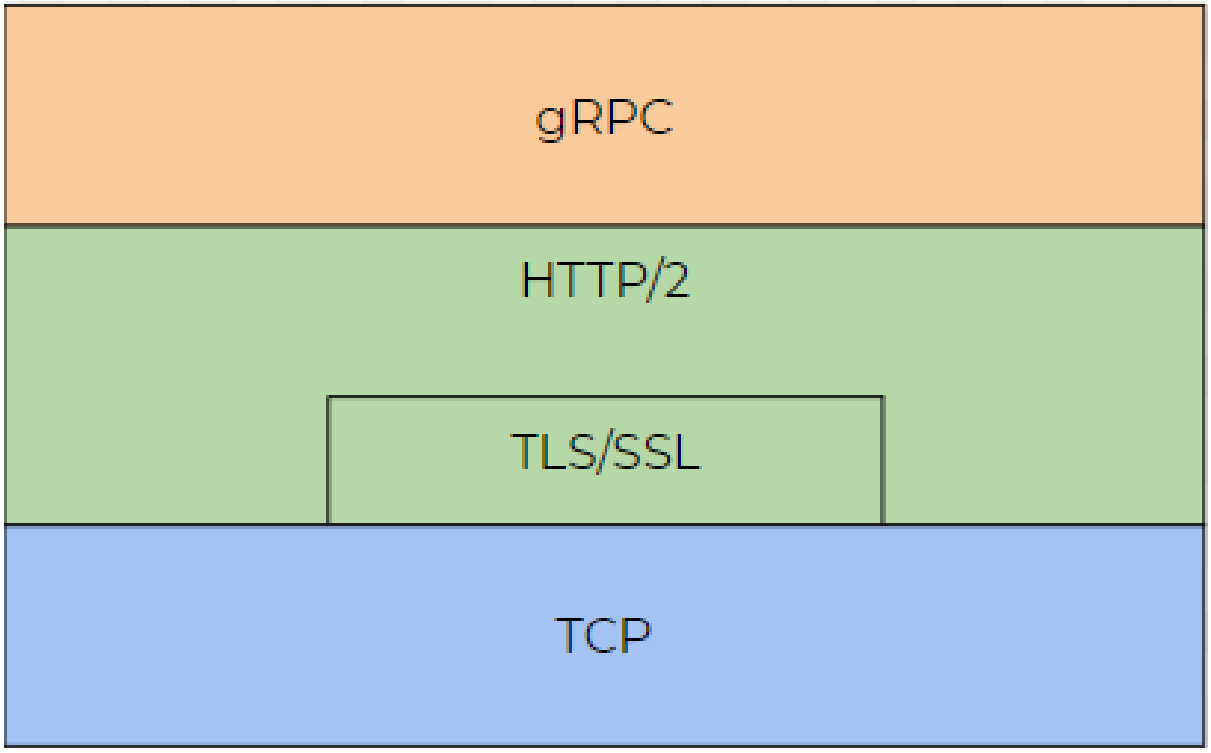


Figure 2: gRPC is built on top of other networking protocols, as shown in the diagram above [15,16]

The clients and the server are connected using this framework. In our case, the SPOT robot is the server, and the client can be either the tablet or any connected PC. Protocol buffers are used to communicate. Protocol Buffers are a language-neutral, platform-neutral extensible mechanism for serializing structured data. Google's mature open-source mechanism for serializing structured data. The protocol buffers are defined using ". proto" extension definition files. The structure for the data is defined in this file. A protocol buffer compiler (protoc) generates data access classes in your preferred language(s) from your proto definition.

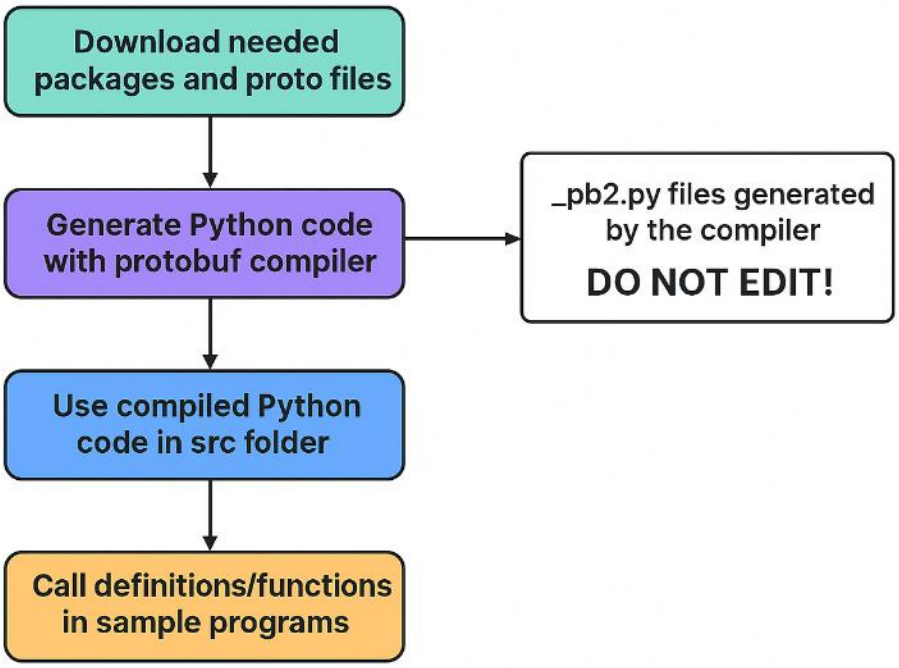


Figure 3: Working with RPC and SDK imports, image generated using ChatGPT

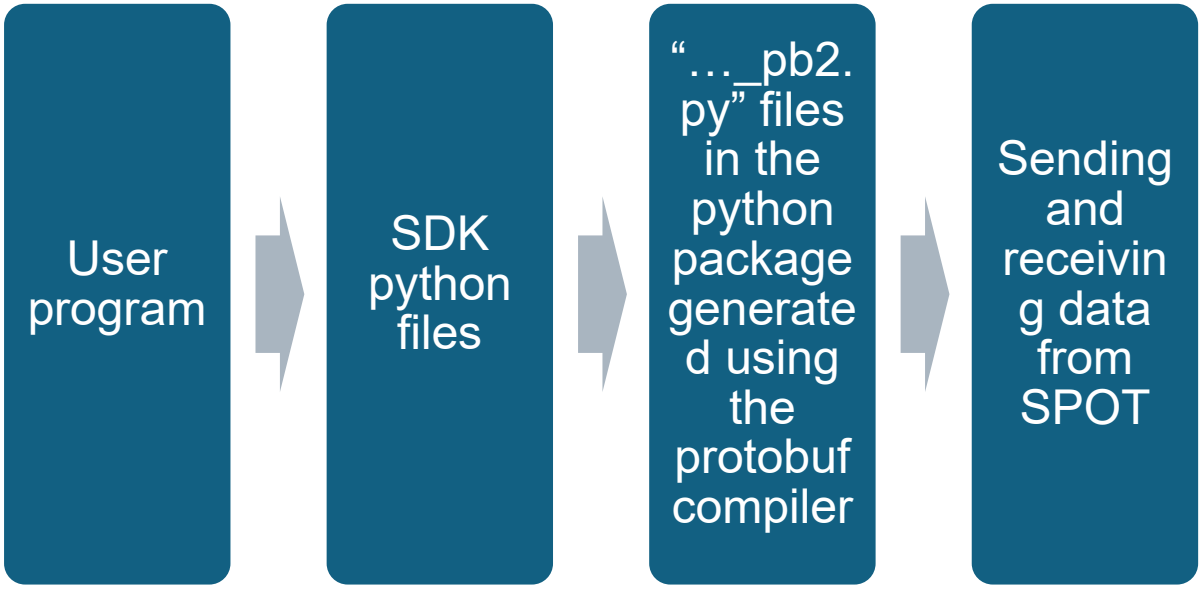


Figure 4: Flow chart of the process at the client end

There are four types of streaming services that gRPC provide. They are listed below, and a combination of these services is used as per the requirement. Different streaming types support varying information exchange. For e.g. Video streaming from Spot to another device can be done with single request at the start and then multiple responses to view a live stream from the cameras. On the other hand, a Unary stream can be used to establish a connection and then a synced bi-directional streaming service can be used for checking the validity of the connection for the duration of operation. Having sync and async communication is also possible to avoid the program from hanging/lagging. [16]

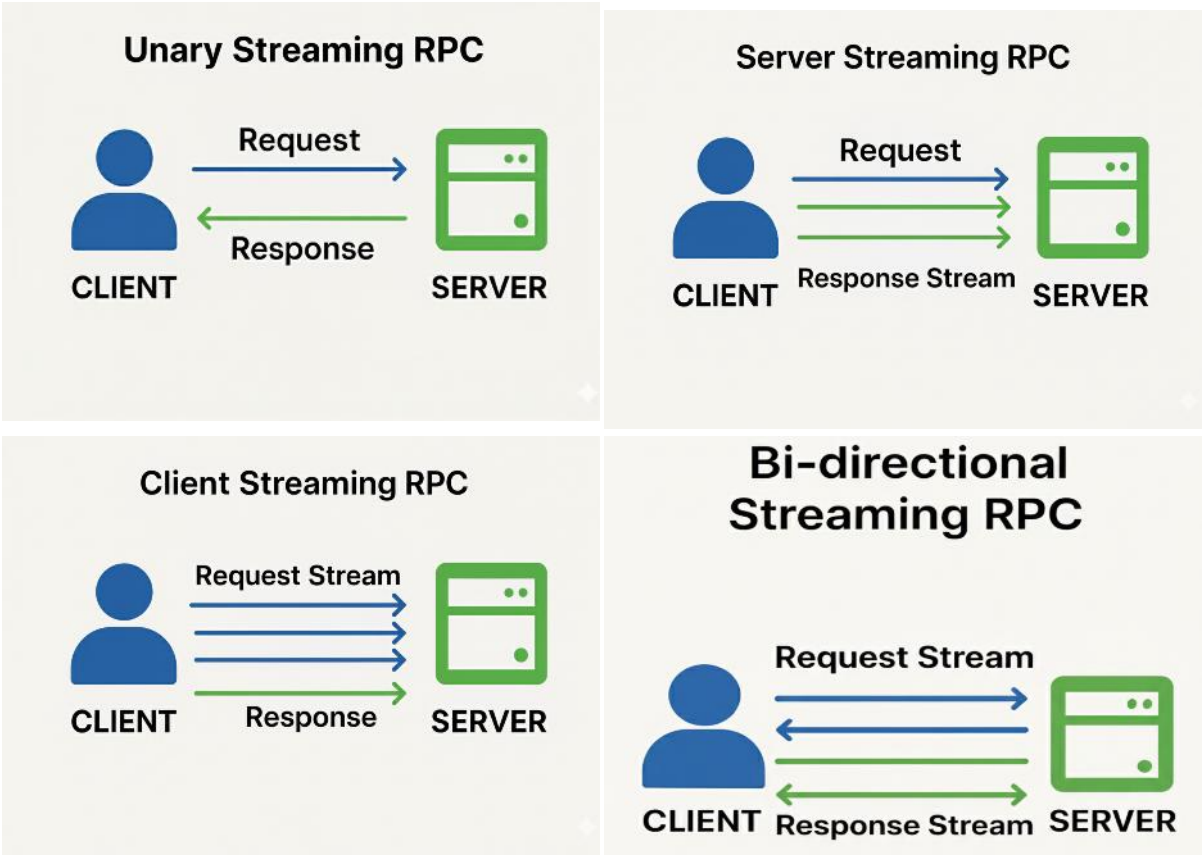


Figure 5: Types of service methods available in gRPC, image generated using ChatGPT

The Spot SDK is made using Python. The commands from the SDK are then converted into language neutral signals using protocol buffer (protobuf) compiler and send to server (Spot) through RPC. This gives the advantage of establishing communication with devices working with different languages at the local level and using different streaming services. [15,16]

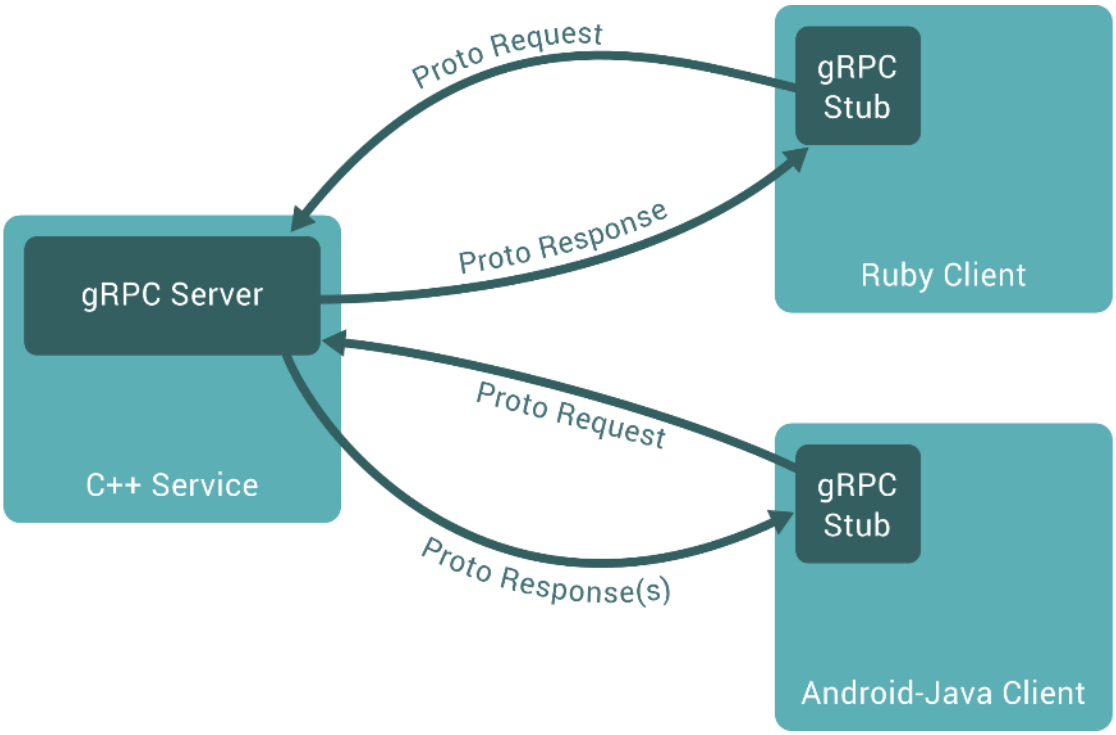


Figure 6: Communication using gRPC with server and client using different languages at local level and also varying streaming services(block diagram) [16]

3.3 Hardware

3.3.1 SPOT

SPOT, designed by Boston Dynamics, is a quick and lightweight autonomous robot with four legs, intended for various scenarios such as industrial monitoring and inspection, public safety verification, and research. It can self-navigate both indoors and outdoors utilizing various onboard sensors and also external attachments (e.g., robotic arms) that allow it to gather data, monitor conditions, conduct inspection and execute additional tasks. [15,17]

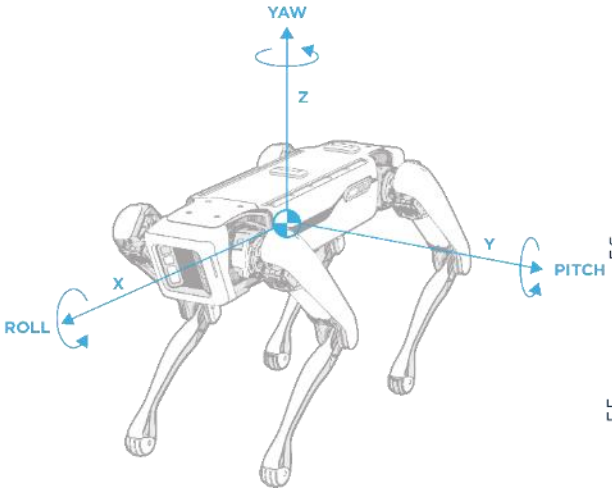


Figure 7: SPOT axes that are used for motion definition[15]

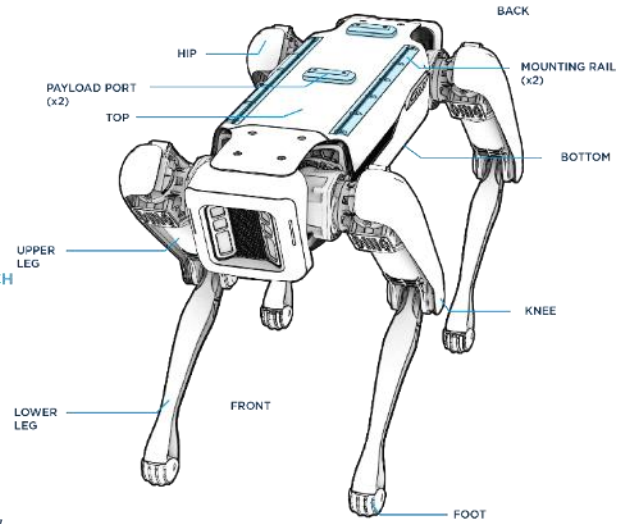


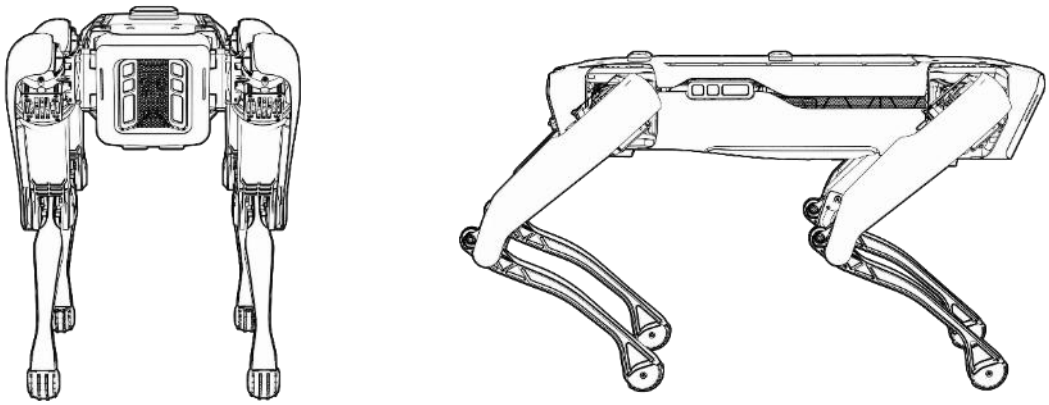
Figure 8: Basic labelled diagram of Spot[15]

Specification	**Value**
Robot type	Spot Gamma
Length	1100 mm (43.3 in)
Width	500 mm (19.7 in)
Height (standing)	840 mm (33.1 in)
Height	(sitting) 191 mm (7.5 in)
Net weight	32.5 kg (71.7 lbs)
Degrees of freedom	12
Maximum speed	1.6 m/s

Table 1: Robot specifications, dimensions [15]

Specification	**Value**
Ingress protection	IP54
Operating temperature	-20C to 45C
Slopes	+/- 30 degrees
Stairways	Stair dimensions that meet US building code standards, typically with 7 in. rise for 10-11 in. run
Max step height	300 mm (11.8 in)
Lighting	Above 2 lux

Table 2: Robot specifications, environment [15]



Front view of SPOT *Side view of SPOT*

Figure 9: Orthographic views of Spot [15]

SPOT is equipped with five integrated cameras, including both fisheye and depth sensors, that allow it to perceive its surroundings. These cameras are made accessible through the robot's internal image service, which supports simultaneous multi-camera image acquisition with hardware-based time synchronization when multiple image sources are requested with a single RPC.

Table ***Error! No text of specified style in document.****-1*

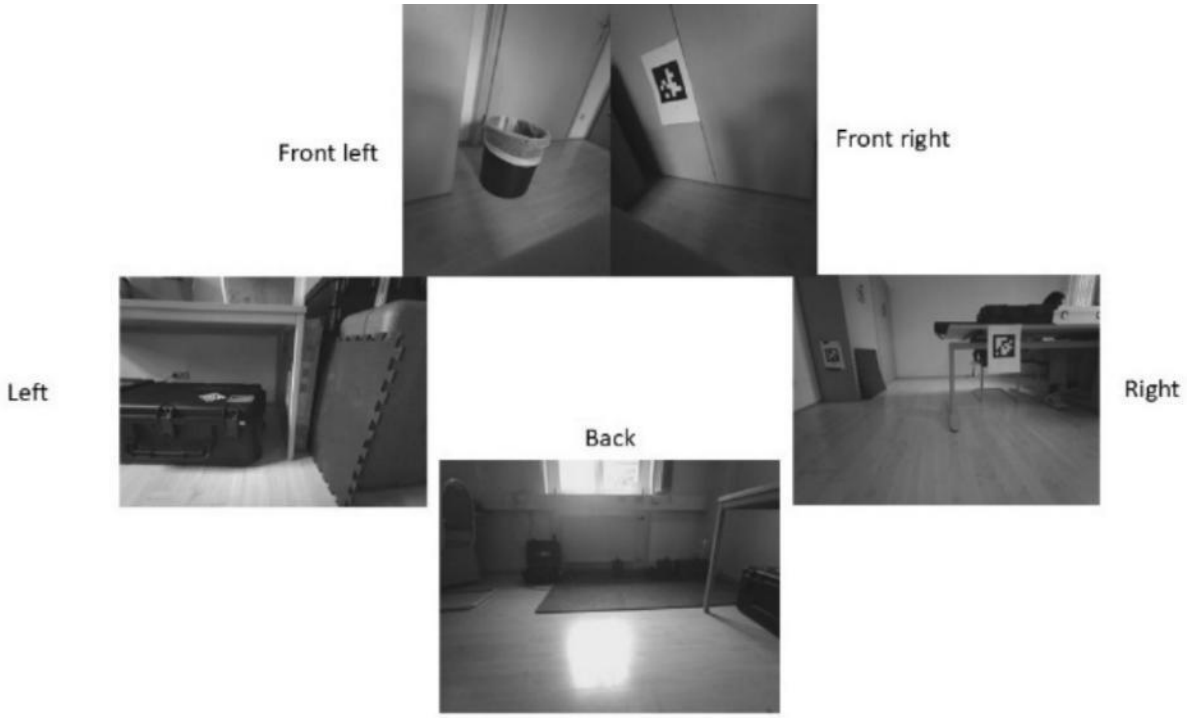


Figure 10: Collective view of all the cameras of SPOT

Specification	Value
Battery capacity	605 Wh
Max battery voltage	58.8V
Typical runtime	90 minutes
Standby time	180 minutes
Charger power	400W
Max charge current	7A
Time to charge	120 minutes

Table 3: Robot specifications, Power [15]

Specification	Value
Max weight	14 kg (30.9 lbs)
Max power per port	150W
Payload ports	2

Table 4: Robot specifications, Payload [15]

Specification	Value
Camera type	Projected stereo
Field of view	360 degrees
Operating range	4 m (13 ft)

Table 5: Robot specifications, Sensing [15]

Specification	Value
Wi-Fi	802.11
Ethernet	1000Base-T

Table 6: Robot specifications, Connectivity [15]

3.3.2 Core I/O

The SPOT Core I/O is an advanced auxiliary computing module engineered by Boston Dynamics to expand the processing capacity and communication performance of the SPOT robotic system. Designed as a durable, field-ready payload, it enables SPOT to function as a computing platform with capabilities for autonomous data analysis, complex sensor integration, and real-time operational decision-making. At its core, the SPOT Core I/O utilizes NVIDIA's Jetson Xavier NX module, which delivers high computational performance within a compact design. [18]

Figure 11: SPOT Core I/O payload [18]

3.3.3 Ricoh Theta Z1 360° camera

The Ricoh theta Z1 360° camera is user friendly for use on The Boston Dynamics SPOT, as it can be connected or integrated through the Core I/O module, or directly through an external client computer. Once connected, the capabilities of the SPOT API and broader integrated software ecosystem may allow for the automated capture of 360° images and video for inspection or documentation purposes. [15]

360° Image and Video Capture: The Ricoh theta Z1 captures immersive, high quality 360° images with a resolution of about 23 Megapixels (6720 x 3360, 7K), as well as 4K (3840 x 1920, 29.97 FPS) video recording capability. The theta Z1 produces high quality, seamless, single panoramic images using advanced image stitching algorithms. The theta Z1 has a 4-channel microphone array for recording high quality audio to fully capture the immersive audio experience. [19]

Figure 12: Ricoh Theta Z1 360° camera, which has the possibility of being used as the payload. [19]

3.3.4 X-Box controller

The Xbox wireless controller is compatible with the Boston Dynamics SPOT robot, offering an intuitive method for manual teleoperation. This functionality is supported by an example in SPOT SDK, which utilizes the XINput API on a Windows system. It enables operators to control SPOT in real time using a familiar gamepad interface, providing significant benefits for rapid setup, operator training, and direct human-supervised field operation. [15,20]

Figure 13: X-Box controller (with wired and wireless functionality) [20]

3.3.5 Client System

A client computer, either a PC or Laptop, is needed to program, control and process data from SPOT through API. This configuration facilitates advanced operational workflows by extending SPOT's autonomous capabilities, delegating intensive data processing to external hardware, and allowing seamless integration within larger automation and robotic systems. [15]

3.3.6 Fiducial tags

The Spot's world object service can recognise QR codes mostly referred to as fiducial tags of the belonging to AprilTags in the Tag36h11 set. The default Image size: 146 mm square. Printed on white non-glossy U.S. letter-size sheets (preferably rigid). For 500-series fiducials used with a Spot Dock, the outer boundary of the fiducial sheet should be 200.8 mm wide x 215.5 mm high. For wall-mounted fiducials, the ideal position is 45 to 60 cm above the ground. The range of detection for a normal size fiducial is 3 meters. However, the size of the fiducial can be adjusted from the SPOT Admin Console. [21]

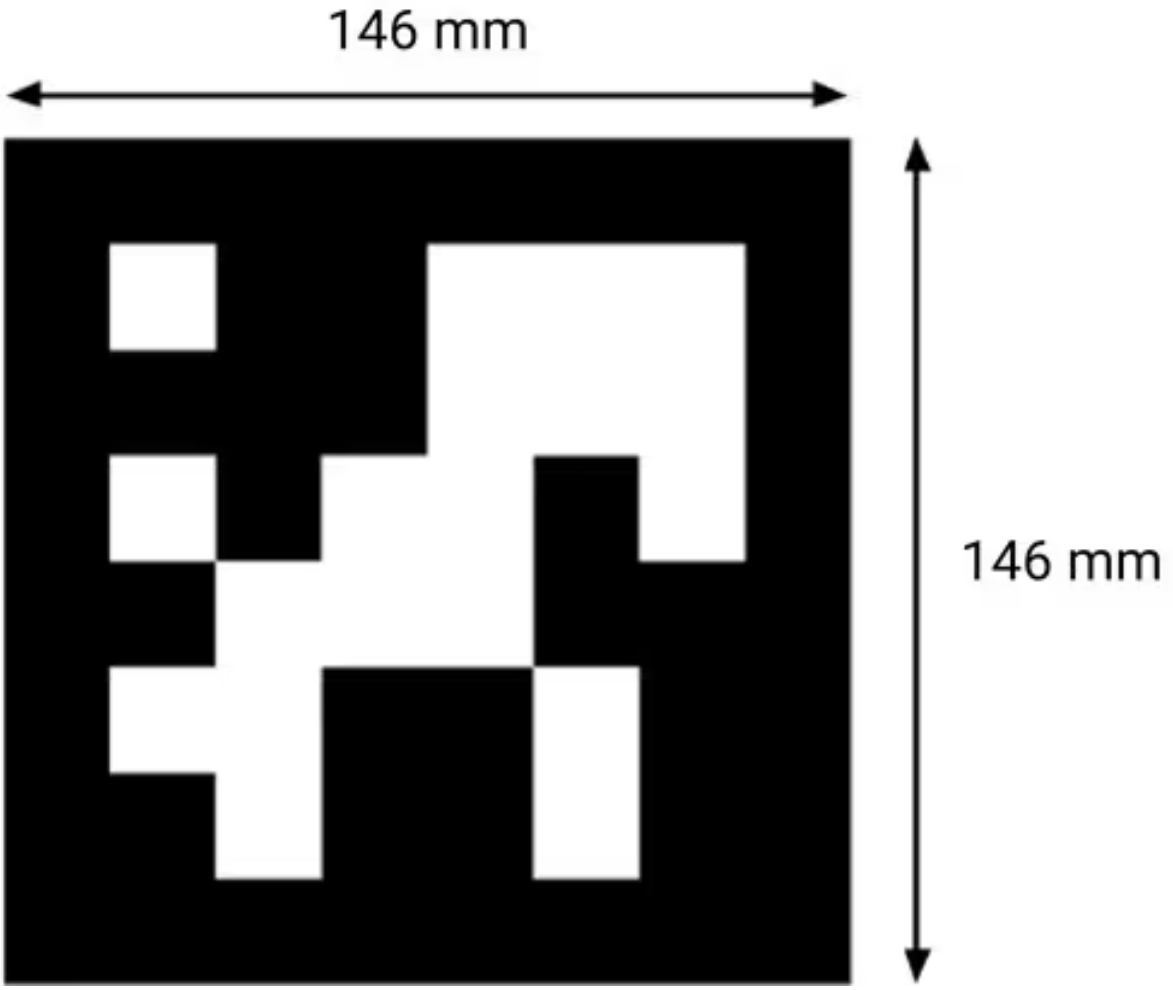


Figure 14: Fiducial tag with dimensions [21]

3.4 Software

3.4.1 Microsoft Visual Studio Code (VS Code)

Visual Studio Code was the IDE used to develop with the Boston Dynamics Spot SDK (Python), and it had a few niche advantages for this workflow. IntelliSense, and provocative "Go to Definition" features for the bosdyn packages allowed rapid exploration of clients, services, and fields of requests and responses, meaning that less time for the programmer was spent with API lookups. Typing and running scripts from the integrated terminal facilitated virtual-environment, installation of SDK wheels, and running scripts -- all within the same workspace, meaning that edit-run time was reduced flat out (it was one action for three conceptual actions). The Python debugger (breakpoints, watches, step-in/over) allowed the programmer to trace gRPC calls, view protobuf messages, and check deadlines/retries/and errors without having to excessively print. Linters and formatters (Pylance/Ruff/Black) supported the programmer's ability to keep their wrappers and other utility functions consistent and readable. Lastly, built into the IDE Git tools streamlined diffs and commits associated with SDK scripts and configs they fed into. [22]

Python 3.10 used for project

3.4.2 GitHub

We utilized GitHub as our external version control system to document changes in code, keep tasks synchronized across machines, and maintain a historically verifiable account of the SPOT SDK scripts over time. Each commit represented a snapshot that recorded incremental changes and accompanying messages that summarized the rationale of the update. We occasionally made use of branches to isolate experimental features and edit the code for possibly merging back into the main line later. Pull-request style reviews (even in a solo workflow) provided a place to compare diffs, discuss changes, and double-check edits were intentional prior to ingestion. Issues and README notes provided documentation of work, configuration hints, and known limitations to help remind each other of our work in open-source spirit of maintaining a self-documenting repository. For this project there were no official releases or tagged versions; stable points in the repository were referenced by commit hashes and protected through the default branch. In this way we aimed to balance tracking with efficiencies - we utilized Git clone or pull commands to bring the most up to date repo on any development host, and if necessary, revert to a past state by checking out the relevant commit. [23]

Git repository link: https://github.com/abhishekdilippatil/spot-sdk

3.4.3 Spot SDK

To implement the robot-side functionality and client application development in the software stack of the project, we used the Boston Dynamics Spot SDK (Python). This SDK provides gRPC-based client libraries to expose certain services-authentication, time sync, lease/ESTOP and power management, robot state, image and data management, mobility and manipulation-to enable repeatable scripting of control workflows (connect → authenticate → obtain lease → power on → issue command → release) while providing high-level helpers and sample programs to simplify many common tasks (this includes controls for commanding base and arm trajectories, subscribing to camera/image streams, recording telemetry, fault recovery, etc.). The SDK defines each of the features as a typed client (e.g., RobotCommandClient, ImageClient), which reap the benefits of a common request/response schema, status codes, and error handling for each feature. The architecture provides everything that the project needs for safe teleoperation, fiducial driven behaviours, and low latency video, while supporting solid session management (deadlines, retries, and leases) when in the field. [15]

Version used for project: 5.0.1

4 Implementation and Test cases

4.1 System Architecture

The system follows a modular client-robot architecture. A lightweight launcher ('run_cmd.py') begins standard startup behaviour by calling the single-instance main controller in a console. The (single instance) controller ('spot_control_manager.py') serves as the coordinator, initializing Spot SDK sessions (authentication and time sync), acquiring and holding safety resources (lease, ESTOP, power), and exposing a curses-based UI for operator teleoperation and monitoring status. Behaviour is encapsulated in 'fiducial_follow.py' which detects fiducial tags using the world-object service, selects a target (it has priority that is 'master' or operator chosen), computes an offset goal and heading, and calls an SE2 trajectory command, an additional viewer streams camera feeds asynchronously for situational awareness. The controller and follower each have their own type of SDK client (RobotCommand, RobotState, Image, WorldObject, Lease, ESTOP, Power) for communicating with the robot via gRPC. Telemetry, logs, and video are streamed from the robot to operator UI; all actuation is gated by lease and ESTOP to provide clear enforcement of safe ownership including immediate stop capability. This separation of concerns (launcher, controller/safety, behaviour, I/O) provides a responsive runtime that is structured for testing and eases behaviour or sensing extension. [24]

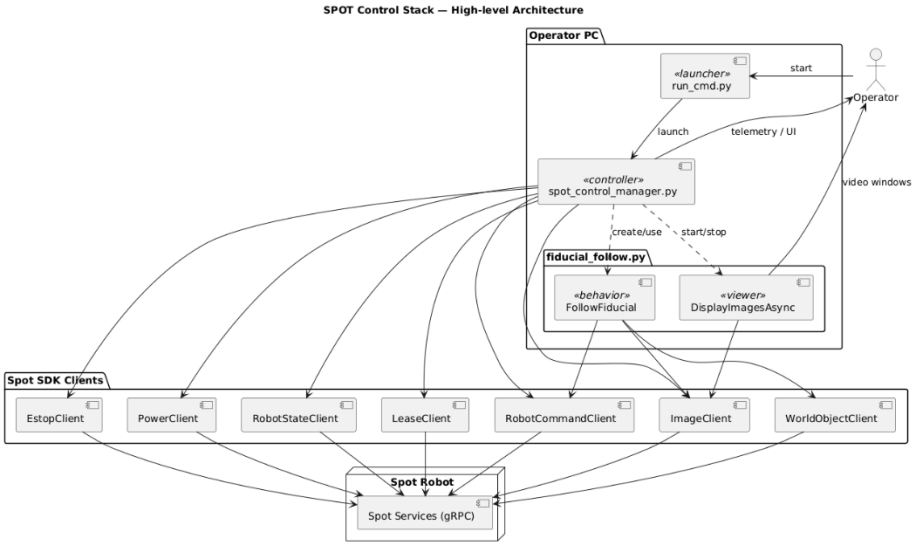


Figure 15: High level System Architecture of the spot program generated using PlantUML [24,25]

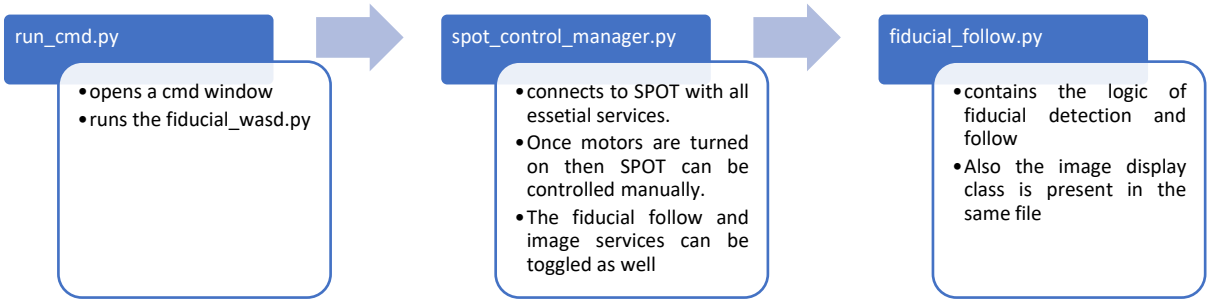


Figure 16: Code Organization. Implemented by considering the Software engineering principles.

4.2 Python program

Program path in the Git repository: spot-sdk\python\self-programs\fiducial_wasd

Repository branch name: “v0”

1. Run_cmd.py

   A minimalist launcher that figures out the directory of the script, change the working directory to there, and opens a new terminal to run spot_control_manager.py with the necessary arguments. It only exists to keep consistent how the main controller is launched, and to view the logs in the new console. [24]

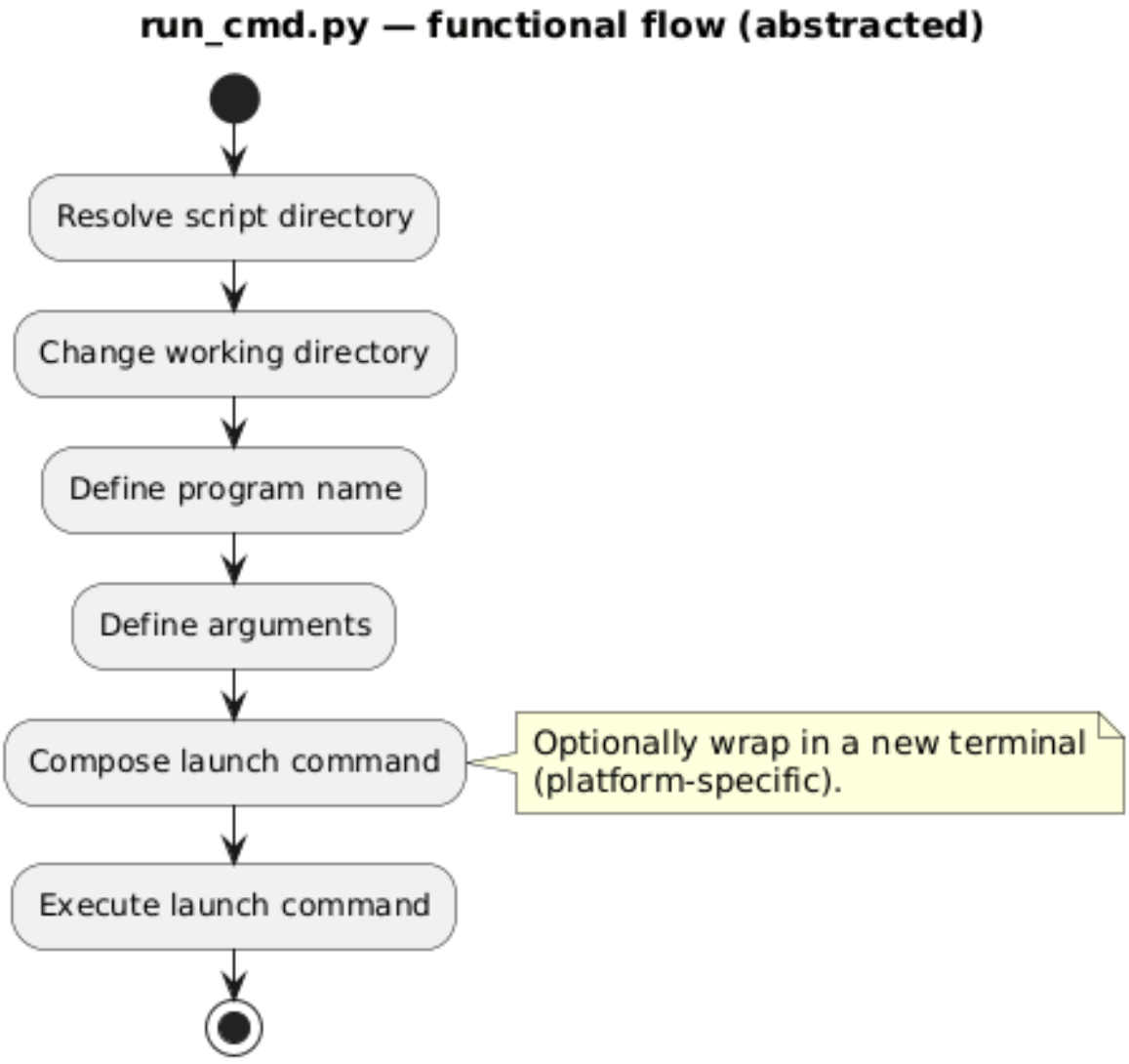


Figure 17: functional flow of run_cmd.py generated using PlantUML [24,25]

2. Spot_control_manager.py

An interactive controller that establishes SDK connections (authorization, time synchronization), handles safety resources (emergency stop, lease, power), and provides a curses-based user interface for teleoperation and status display. Inside a timed control loop, it displays the robot state, reads keys, and sends actions (stand/sit/stop, moves at velocity, camera view, start/stop following fiducials) before shutting down cleanly by returning the lease and stopping auxiliary threads. [24]

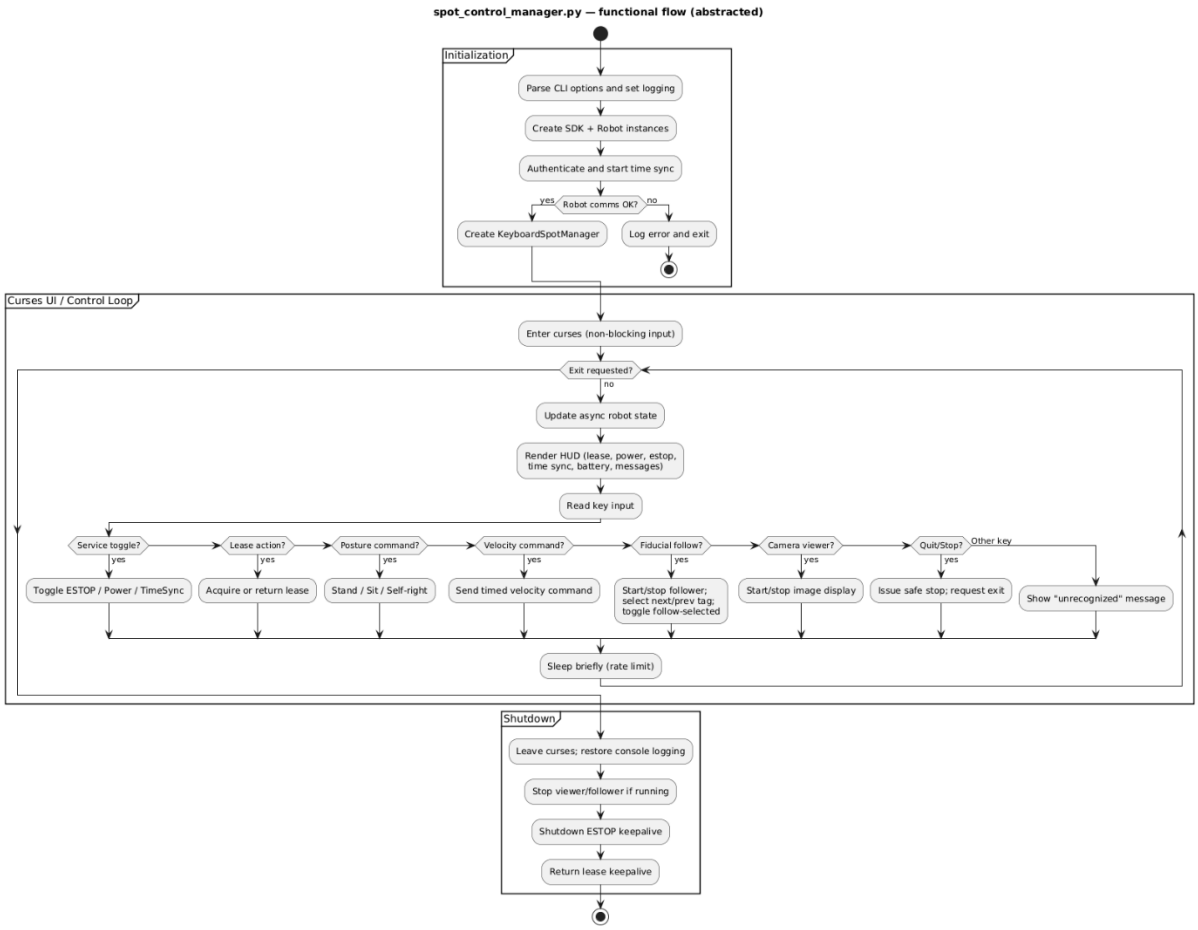


Figure 18: functional flow of spot_control_manager.py generated using PlantUML [24,25]

Keys	Feature	Explanation
TAB	Quit	Quits the program immediately
T	Time-sync	Time syncs the client and robot time
SPACE	E-Stop	Toggles E-Stop
p	Power	Toggles motor power (Motor power can only be toggled if E-Stop is not activated)
f	Stand	Spot stands
r	Self-right	Spot turns back up from battery change position
wasd	Directional strafing	For movement in four directions
v	Sit	Spot sits
b	Battery-change	Spot tilts and rests on two legs, allowing battery to be changed easily. Note: Spot turns off while exchanging batteries
qe	Turning	For turning the Spot
i	Camera view	Toggles the video stream
ESC	Stop	Stops the robot immediately at place.
o	Toggle fiducial follow	Spot starts to detect fiducials and can follow the master tag
l	Return/Acquire lease	Toggles lease
m	Next fiducial code	Cycle through code list in forward direction
n	Previous fiducial code	Cycle through code list in backward direction
x	Toggle follow current fiducial ID	A toggle switch to follow/ unfollow selected fiducial

Table 7: Key bindings for manual input [24]

3. Fiducial_follow.py

Perception-driven behaviour module that recognizes AprilTag fiducials through Spot's world-object service, selects a target (master tag or operator selected ID), computes target goal and offset heading, and issues commands for SE2 trajectory motions with optional speed limiting and obstacle avoidance settings. It also supports an asynchronous image viewer for the chosen cameras, and utilities for starting/stopping following, cycling through selection, and stopping motion immediately.[24]

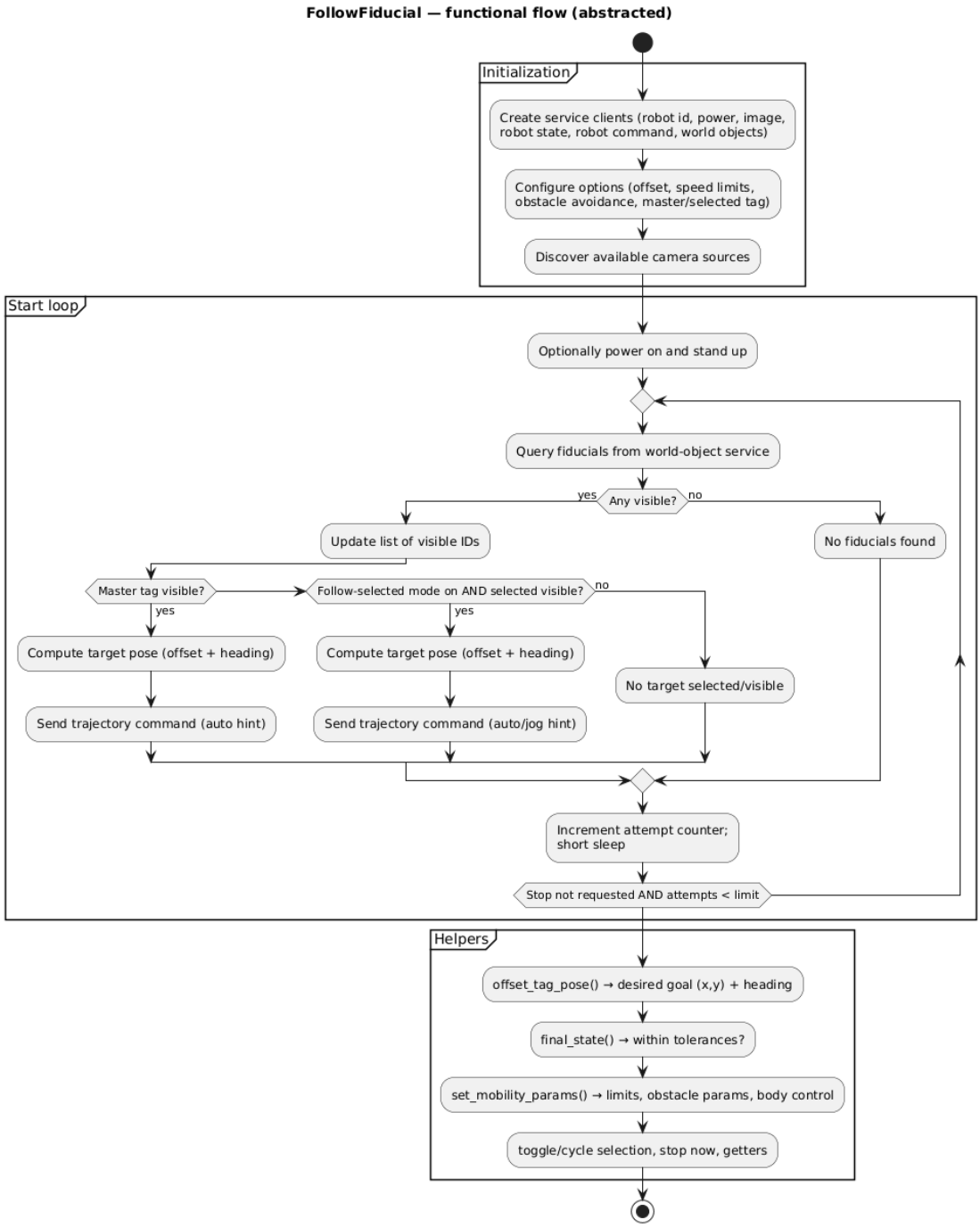


Figure 19: functional flow of fiducial follow class generated using PlantUML [24,25]

Certain fiducial IDs are linked to "special behaviours" that help simplify and speed operation and recovery behaviour. A particular master tag is assigned the highest priority - when it is seen, it forces the overriding selection of all others and to initiate a default follow behaviour, with a safety approach offset and heading alignment. The other tagged IDs are mapped to specific actions - for example, switching from a fast jog locomotion hint to a slower jog in tighter space or moving directly to a short inspect dwell with streamed video being stable, or switching on a discrete digital signal to the client for the logging/automation of external functionality. These behaviours will be gated by safety interlocks (lease/ESTOP) and timeout processes, so they will never suppress the Manual Override input - operator inputs will always override tag actions. The tag → action mapping is table driven, it is easy to add, remove, or retune behaviours without changing code. [24]

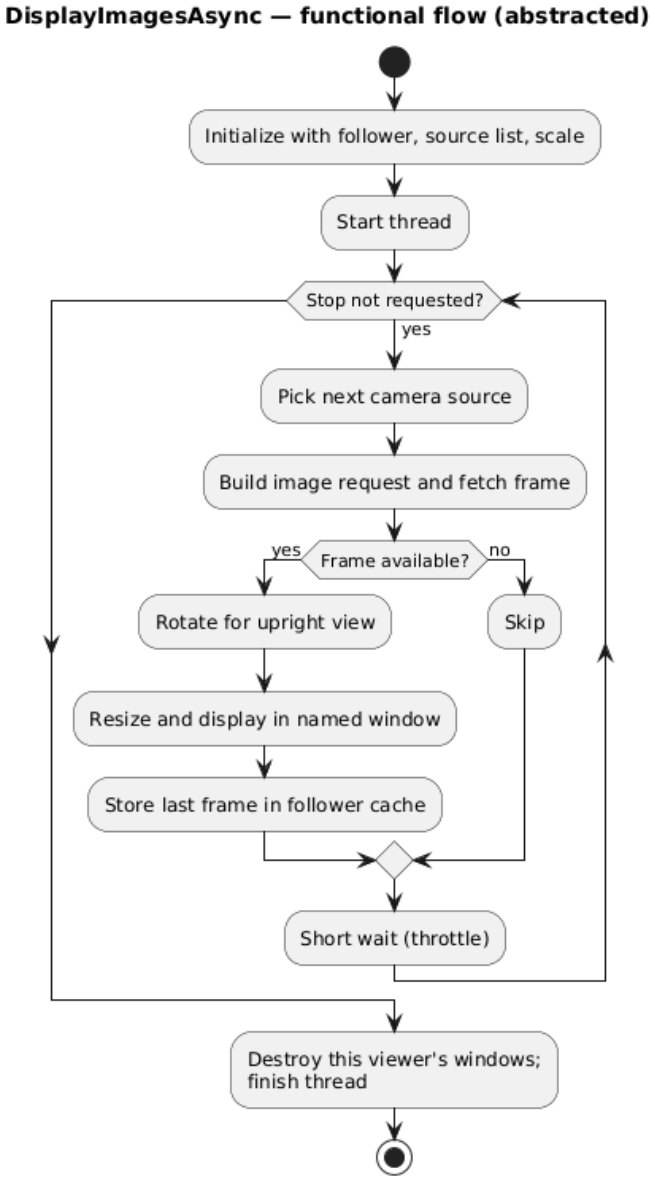
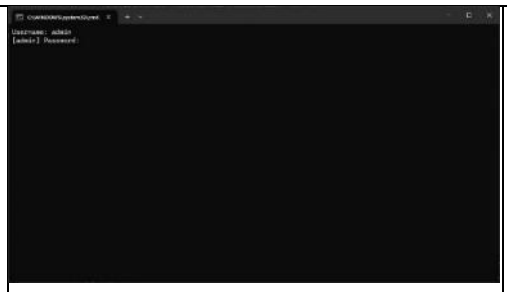
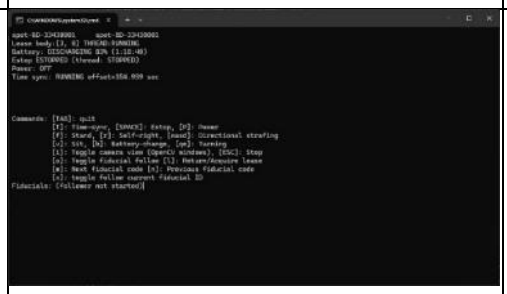
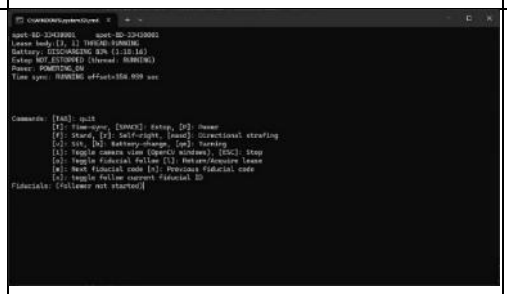
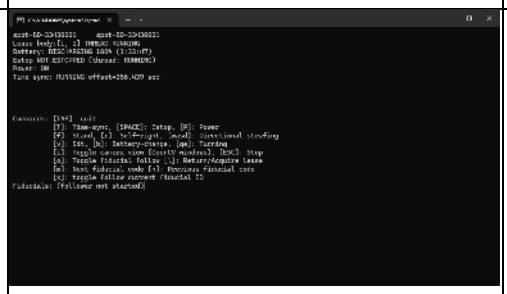
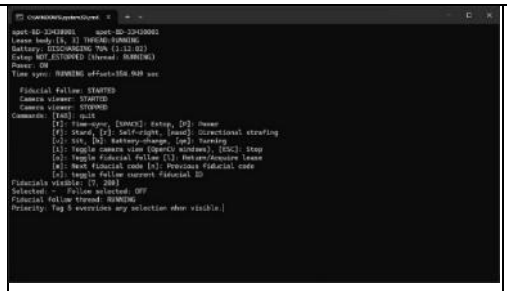
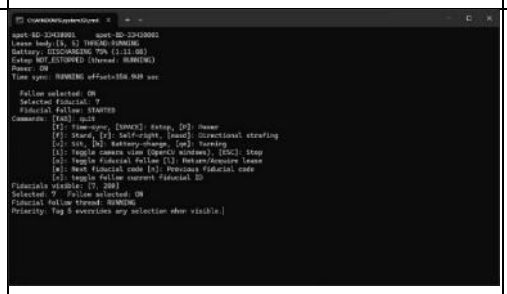
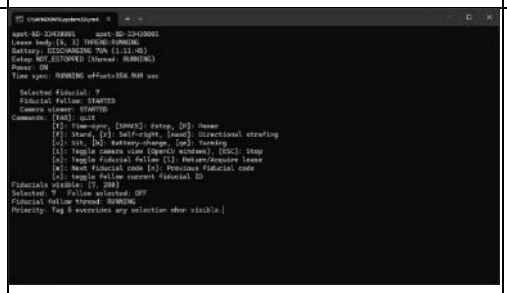


Figure 20: Functional flow of image streaming class generated using PlantUML [24,25]

4.3 Basic working of the Program

Below is a step-by-step depiction of the basic functions of the program

1		 Connecting to Spot with username and password.
2		 Checking lease connection and Time sync.
3		 Disengaging E-Stop and turning Motor power ON.
4		 Manually standing up the robot.

5		 Scanning for fiducials
6		 Selecting the fiducial  Walking towards the fiducial
7		 Manually bringing back Spot and starting to power down procedure in the reverse order as done during powering up.

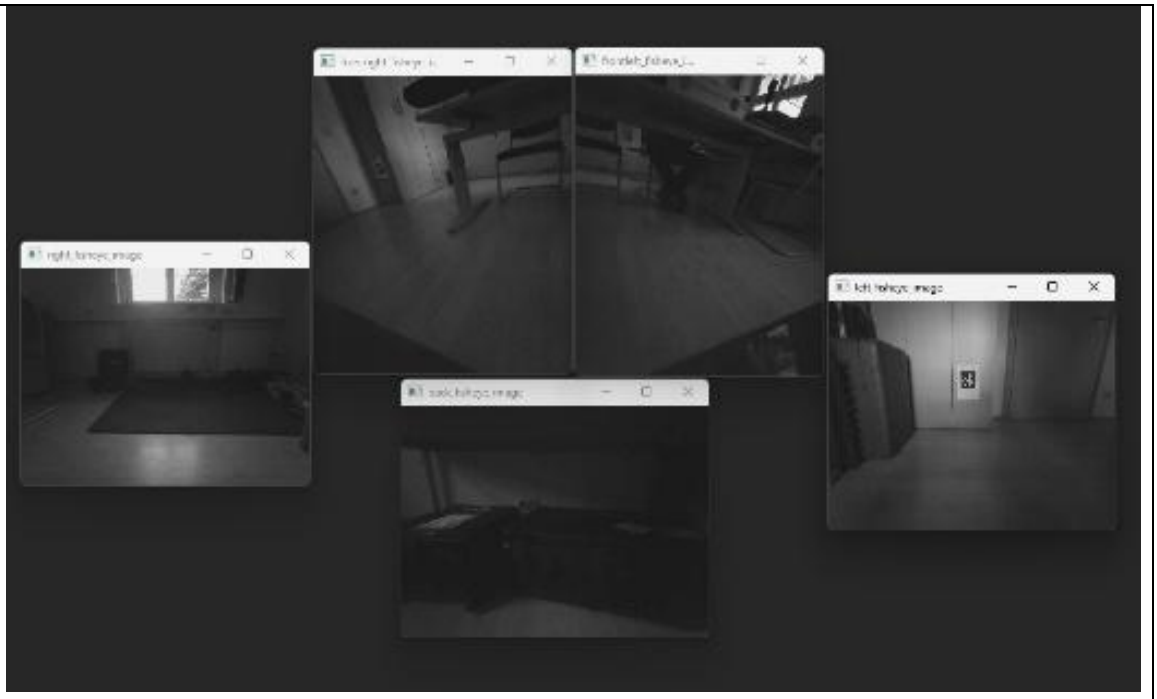
8	 Camera feed snippet

Table 8: basic function flow of the program

Note: The world object service keeps the info of the tag even after the tag is removed from the Spot vision for about 7~15 sec. This is done to not lose the Fiducial info while navigating towards one and accidentally, loosing data about the fiducial as it vanishing from the visible area. [15]

4.4 Test cases

Additional test cases were also researched in the duration of the project.

1. Detecting fiducials using at the client end using AprilTag Library

   The first trail for detecting fiducial tag was implemented by using the open source AprilTag Library. In this case only the camera feed was received from the Spot as raw data. Then the client PC ran an algorithm to detect fiducial in the incoming stream. After that the distance and the heading were calculated with the help of the library. And then the movement command was given to Spot accordingly. The drawbacks were, improper heading, doggy movement. Laggy video stream due to huge amount of data being sent through the network for analysis. Hence, it was better to stick with the in-built "world object detection" of Spot for identifying the tags and the robot "get_vision_tform_body" service is used to transform the coordinates from the vision frame to the body frame. This resulted in highly accurate positioning and less load on the client PC and the network.

2. Possible to descend stairs in manual mode, but tricky in fiducial follow

   Spot has an inbuilt flag which detects the stairs in its surroundings. By default, this is true. While ascending the stairs, the heading of the robot and the direction in which it can ascend is the same (forward direction). Hence Spot can ascend stairs in fiducial follow mode as well. Contrary to this, Spot can only descend the stairs with hind legs first. i.e. in backward direction. A preliminary code was tested with the Spot where the heading used to dynamically change to back_fish_eye. When stairs were detected. However, if the robot stops in the middle of the stairs. It finds difficult to acknowledge that it is still positioned on the stairs. The robot has been observed to change the heading back to front direction in midway. Hence, more work and time is needed to come up with a robust code that can handle such complex situations for autonomous fiducial follow. On the other hand, during manual control the operator can simply turn the robot around and descend the stairs if needed. Hence, proving the benefit of having manual control along with autonomous behaviour.

3. Usage of Core I/O for more advance interaction with surroundings

   Core I/O can increase the computational power of Spot hence allowing more complex sensors to be mounted with Spot where the data is first partially stored and processed in Core I/O before sending it to the client. The application of Core I/O is with heavy payload like scanners and high-definition cameras to handle the huge data without interfering with the core features of Spot. Unfortunately, due to technical problems with the Core I/O further research was inhibited and only theoretical research was done. During research, it was found that custom programs can be uploaded on the Core I/O and deployed with the help of Docker system. This can help to perform high computational task on the Spot body itself withing Core I/O keeping the communication

between client and Spot clear. This can help in features such as image/ object recognition, machine learning, etc. [15]

4. Testing with other equipment such as Ricoh Theta Z1 360° camera and Controllers

   While using Ricoh Theta with Spot is a common example as there have been applications before in case-studies where Spot was mounted with high-definition cameras for inspection. It made us curious to understand how a camera can be integrated with the Spot. We were able to integrate it with the client PC over the same network as Spot.

   Also, the controller was tested to control Spot manually with the help of "XInput" library for windows PC. Both the tests were success, but due to time limitation of the project, this has been kept open for further work.

4.5 Results and Discussion

The implemented program is robust and capable of navigating with the help of fiducial ID detection. Successful distinction between different IDs is possible which can also help to make Spot react to certain tags. The control manager ensures a single and secure connection is established to avoid any gRPC errors and timeouts.

The Spot is always interacting with the surrounding, scanning for fiducials and responding what is visible to the operator. Then decisions can be made accordingly for navigation. The camera feed also provides a safety feature for the operator to look at the world through Spot eyes. When the need is felt, the operator can guide Spot through difficult-to-navigate terrain. As the feedback loop is still not as fast as an actual person. But the ability to work in hazardous environment and carry decent amount of load is what appeals for the usage of Spot.

The additional test cases performed in course of the project compliment the base of the program and with additional payloads, it can be brought closer to a real interaction between an organism and the surrounding.

5 Conclusion and Future Work

5.1 Conclusion

This project presents a functional control stack for Boston Dynamics Spot that integrates autonomous, fiducial-driven behaviours with continuous manual override with safety constraints. The architecture – a lightweight launcher, a curses-based control manager, and a perception-driven fiducial follower – enabled reliable detection of fiducials and prioritized following of a “master” fiducial, low-latency situational awareness to the operator, and gated actuation through ESTOP, lease, and power services. The field tests yielded stable teleoperation, multiple accurate tag approaches, and clear operator feedback, which were successful with computational and network load ease.

Comparative testing framed important design decisions. The AprilTag pipeline on the client-side was replaced with Spot's world-object service and get_vision_tform_body, providing improved pose fidelity and reduced bandwidth/CPU costs; stair traversal further highlighted the operational benefit of instant manual pre-emption, for edge-cases where autonomous descent remains quite brittle; and early exploration of integrations (Core I/O concepts, 360° imaging, and gamepad teleop) confirmed that richer payloads, and on-robot compute, are feasible when situational constraints permit. Collectively these findings ascribe with recent literature that encourages an end-to-end, iterative approach to reality capture and analysis, instead of capturing alone.

In the end, the framework fills a practical niche between scripted autonomy and real-world variability on construction sites: autonomy continues to operate but allows operator intent to safely take precedence at any moment. The frame of code organization, patterns of SDK use, and UML views provide a reproducible starting point that can be built upon with more behaviours, payloads, and analytics pipelines to enable deployments using more sophisticated perception and BIM-linked progress monitoring in the future.

5.2 Future Work

Thanks to advanced sensors, edge processors, and onboard processing architectures such as the Core I/O, Spot's autonomy can evolve in a significant and complex fashion, while maintaining human oversight. Exploration may further engage AI strategies for perceptive capabilities, as well as machine learning strategies for detecting objects, understanding the terrain, and reflexive path behaviours, to allow the Spot to make intelligent, context-aware decisions, on the fly. Further augmented sensor fusion of LiDAR, 360° cameras, and fiducial detection could enable robust localization in visually severed situations to reduce reliance on previously marked locations.

Another interesting direction may be the cloud-based collaboration of BIM in which Spot's inspection data is uploaded in real-time, analysed, and presented within digital twin platforms that align and incorporate field data into project models. This framework

will open opportunities for engineering teams to monitor project transitions, quality assurance/quality control techniques to determine changes, and activate contingency plans if needed, all remotely in real-time.

Finally, the study could examine human-robot teaming frameworks possibly related to Spot working alongside a worker in a natural manner, possibly invoking elements of voice commands, gestures and AR visualization. The continuous development of safety protocols, fail-safe protocols and context-sensitive human supervision will be critical to trust and reliability when deploying field crews.

The end goal is to combine human intuition with robot precision in a flexible and adaptable workflow that improves productivity, safety, and digitalization of the inspection process in construction.

6 References

[1] Navon, R., Sacks, R.: 'Assessing research issues in Automated Project Performance Control (APPC)', *Automation in Construction*, 2007, **16**, (4), pp. 474–484

[2] Halder, S., Afsari, K., Chiou, E., Patrick, R., Hamed, K.A.: 'Construction inspection & monitoring with quadruped robots in future human-robot teaming: A preliminary study', *Journal of Building Engineering*, 2023, **65**, p. 105814

[3] Kopsida, M., Brilakis, I., Vela, P.A., eds.: 'e: A Review of Automated Construction Progress Monitoring and Inspection Methods' (2015)

[4] Halder, S., Afsari, K., Serdakowski, J., DeVito, S.: 'A Methodology for BIM-enabled Automated Reality Capture in Construction Inspection with Quadruped Robots', *ISARC Proceedings*, 2021, **2021 Proceedings of the 38th ISARC, Dubai, UAE**, pp. 17–24

[5] Afsari, K., Halder, S., Ensafi, M., DeVito, S., Serdakowski, J.: 'Fundamentals and Prospects of Four-Legged Robot Application in Construction Progress Monitoring'. ASC 2021. 5th Annual Associated Schools of Construction International Conference, California State University, Chico, 274-263

[6] 'Critical Infrastructure: Spot Inspects Hamburg Bridge | Boston Dynamics', https://bostondynamics.com/case-studies/critical-infrastructure-spot-inspects-hamburg-bridge/, accessed 28-09-25

[7] 'Turner Uses Robots to Cut Inspection Time Over 95 Percent | Boston Dynamics', https://bostondynamics.com/case-studies/turner-construction-uses-ground-robots-to-cut-inspection-time-by-over-95-percent/, accessed 28-09-25

[8] 'RATP: Inspections after dark | Boston Dynamics', https://bostondynamics.com/case-studies/ratp/, accessed 28-09-25

[9] 'Virginia Tech | Boston Dynamics', https://bostondynamics.com/case-studies/virginia-tech/, accessed 28-09-25

[10] 'Acciona | Boston Dynamics', https://bostondynamics.com/case-studies/acciona/, accessed 28-09-25

[11] 'Pomerleau | Boston Dynamics', https://bostondynamics.com/case-studies/pomerleau/, accessed 28-09-25

[12] 'Brasfield & Gorrie | Boston Dynamics', https://bostondynamics.com/case-studies/brasfield-gorrie/, accessed 28-09-25

[13] 'Foster + Partners | Boston Dynamics', https://bostondynamics.com/case-studies/foster-partners/, accessed 28-09-25

[14] 'Swinerton | Boston Dynamics', https://bostondynamics.com/case-studies/swinerton/, accessed 28-09-25

[15] 'Spot SDK — Spot 5.0.1 documentation', https://dev.bostondynamics.com/readme#, accessed 29-09-25

[16] ‘gRPC’, https://grpc.io/, accessed 29-09-25

[17] ‘Spot’, https://support.bostondynamics.com/s/topic/0TOUS0000003syL4AQ/spot, accessed 29-09-25

[18] ‘Spot Core I/O | Boston Dynamics’, https://bostondynamics.com/products/payload/spot-core-io/, accessed 29-09-25

[19] ‘RICOH THETA Z1’, https://thetaz1.com/en/, accessed 29-09-25

[20] ‘Xbox Wireless Controller | Xbox’, https://www.xbox.com/en-US/accessories/controllers/xbox-wireless-controller, accessed 08-10-25

[21] ‘About Fiducials’, https://support.bostondynamics.com/s/article/About-Fiducials-77114, accessed 09-10-25

[22] ‘Python in Visual Studio Code’

[23] ‘About Git - GitHub Docs’, https://docs.github.com/en/get-started/using-git/about-git?utm_source=chatgpt.com, accessed 09-10-25

[24] ‘abhishekdilippatil/spot-sdk: Spot SDK repo’, https://github.com/abhishekdilippatil/spot-sdk, accessed 29-09-25

[25] ‘PlantUML Web Server’

[26] ChatGPT: ‘ChatGPT’

[27] Feng, C., Linner, T., Brilakis, I.*, et al.*, eds.: ‘Proceedings of the 38th International Symposium on Automation and Robotics in Construction (ISARC)’ (International Association for Automation and Robotics in Construction (IAARC), 2021)

Appendix

Preparation steps to work with Spot and Spot SDK

1. Initiation Checklist
   - Clear the area & brief people nearby.
   - Ensure an operator has a reachable hardware or software E-Stop.
   - Power and battery check.
   - Put the tablet or your control laptop on the same network as the robot.
2. Programming Operations
   1. **Install the Python SDK**
      1. Install Python 3.7–3.10.
      2. Verify your python install is the correct version. Open a command prompt or start your python IDE:
         Then enter:

```
$ python --version
Python 3.10.12
```

      3. **Windows users:** There are two common methods for starting the Python interpreter on the command line. First, launch a terminal: Start > Command Prompt. At the command prompt, enter either:

```
> python.exe
```

         Or

```
> py.exe
```

   2. **Pip Installation**

      Pip is the package installer for Python. The Spot SDK and the third-party packages used by many of its programming examples use pip to install.
      Check if pip is installed by requesting its version:

```
$ python3 -m pip --version
pip 19.2.1 from <PATH_ON_YOUR_COMPUTER>
```

      For Windows users:

```
> py.exe -3 -m pip –version
```

3. **Install Spot Python packages**

With python and pip properly installed and configured, the Python packages are easily installed or upgraded from PyPI with the following command:

```
$ python3 -m pip install --upgrade bosdyn-client bosdyn-mission bosdyn-choreography-client bosdyn-orbit
```

Installing the `bosdyn-client`, `bosdyn-choreography-client` and `bosdyn-mission` packages will also install `bosdyn-api` and `bosdyn-core` packages with the same version. The command above installs the latest version of the packages.

4. **Verify your Spot packages installation**

Enter this command to verify all the packages

```
$ python3 -m pip list --format=columns | grep bosdyn
```

The Output Should be:

```
bosdyn-api                          5.0.1
bosdyn-choreography-client          5.0.1
bosdyn-choreography-protos          5.0.1
bosdyn-client                       5.0.1
bosdyn-core                         5.0.1
bosdyn-mission                      5.0.1
bosdyn-orbit                5.0.1
```

For Windows Users:

```
> python3 -m pip list --format=columns | findstr bosdyn
```

If you don't see the above Bosdyn packages with the target version, something went wrong while the installation

For more accurate steps and troubleshooting, you can refer to: https://dev.bostondynamics.com/docs/python/quickstart

If packages are not installed successfully, you may see an error:

```
import bosdyn.client
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'bosdyn.client'
```

3. Starting SPOT

    1. Power on SPOT by holding the power button down until the fans start. Wait for the fans to turn off (May take 2-3 minutes)
    2. Connect to SPOT via Wi-Fi
    3. Open the command prompt and ping SPOT at 192.168.80.3
    4. You will have to enter the given username and password each time before sending a command to SPOT

```
$ ping 192.168.80.3
```

    5. Issue the ID command to get SPOT's ID:

```
$ python3 -m bosdyn.client 192.168.80.3 id
```

The Output should give SPOT's unique serial number, it's Nickname, the software version and install date.

Example:

```
beta-BD-90490007     02-19904-9903   beta29     spot (V3)
Software: 2.3.4 (b11205d698e 2020-12-11 11:53:12)
Installed: 2020-12-11 15:06:57
```

If you get this output, then you have successfully communicated with SPOT via Python.

If you get different messages,

Example:

```
$ python3 -m bosdyn.client 192.168.80.3 id
Could not contact robot with hostname "192.168.80.3"

$ python3 -m bosdyn.client 192.168.80.3 id
RetryableUnavailableError: _InactiveRpcError: gRPC service
unavailable. Likely transient and can be resolved by
retrying the request.
```

The robot is not powered on or is not reachable. Go back to Pinging SPOT.

4. Get copy of Full SDK distribution from GitHub

Users can either:

1. git clone https://github.com/boston-dynamics/spot-sdk.git (recommended)
2. Download a zip file distribution:
   a. Select green box "Clone or download" from the webpage.
   b. Select "Download ZIP".
   c. Unzip the file to your home directory.
   d. Rename the top-level directory `spot-sdk-master` to `spot-sdk`. (only for consistency with this document, not required)

5. Run an independent E-Stop

Useful Hint: While working with any SPOT example, always use the given `requirements.txt` to install dependent third-party packages

Before SPOT does anything, we must ensure that an independent E-Stop is running.

Change working directory to E-stop example, do a Pip install with `requirements.txt` as an argument so that any dependent packages are installed and then run E-Stop nogui version.

```
$ cd ~/spot-sdk/python/examples/estop # or wherever you installed Spot SDK

$ python3 -m pip install -r requirements.txt # will install dependent packages

 $ python3 estop_nogui.py 192.168.80.3
```

You should now have a big red STOP button displayed on your screen. You're now ready to go! (or stop in an emergency!!)

6. Run Hello SPOT

   Now we have E-stop running, open another command window, and again run hello_spot:

```
$ cd ~/spot-sdk/python/examples/hello_spot # or wherever you installed Spot SDK
$ python3 hello_spot.py 192.168.80.3
```

   SPOT should power on motors, stand up, give some poses, take a picture and sit down.

   If it did not, make sure the **Motor Power** is enabled (red glowing button near power button).

   If it did, you are a SPOT programming Example Operator.

7. Next Steps

   1. Read Understanding Spot Programming — Spot 5.0.1 documentation.
   2. Take time to explore programming examples, almost all of them can be launched with same procedure as hello_spot.
   3. If something is unclear go through QuickStart — Spot 5.0.1 documentation